



Applied Project 1: A Comparative Study of Deep RL Algorithms

Reinforcement Learning, EE-568

DQN, PPO, SAC and TD3 on Four Classic-Control Benchmarks

Student 1: Kevin Abou Jaoude, 358300

Student 2: Youssef Dib, 339964

Student 3: Fuad Khuri, 357574

May 15, 2026

1 Introduction

This project re-implements four canonical deep reinforcement learning (RL) algorithms from scratch in PyTorch and compares them across four Gymnasium classic-control environments. The algorithms span the two main axes of the deep RL taxonomy: value-based versus policy-based, and on-policy versus off-policy. Deep Q-Networks (DQN) [6] is value-based and off-policy; Proximal Policy Optimization (PPO) [7] is policy-based and on-policy; Soft Actor-Critic (SAC) [4] is policy-based and off-policy with maximum-entropy regularization; Twin Delayed DDPG (TD3) [3] is deterministic, policy-based, and off-policy. The four environments are CartPole-v1 and Acrobot-v1 (discrete actions), and Pendulum-v1 and MountainCarContinuous-v0 (continuous actions) [8]. We run three random seeds per (algorithm, environment) pair, report bootstrap 95% confidence intervals over seeds, and accompany every quantitative claim with an entry in a unified scoreboard table.

We preview four empirical headlines:

1. On CartPole, DQN is highly seed sensitive (a textbook bimodal failure), whereas PPO is reliably stable.
2. On Acrobot, the discrete-action picture reverses: DQN solves the task cleanly in roughly 30k steps, while PPO is slower and unstable.
3. On dense-reward continuous control (Pendulum), SAC and TD3 are roughly an order of magnitude more sample efficient than PPO.
4. On sparse-reward continuous control (MountainCarContinuous), the off-policy methods fail to escape a lazy local optimum, and only PPO ever discovers the goal.

The brief explicitly asks for honesty about the implementation details that turned out to matter in practice. We therefore devote a full page to an implementation diary that records the hy-

perparameter sweeps, the subtle bugs, and the design decisions that distinguish a paper recipe from a working implementation.

2 Background

We model each environment as a discrete-time Markov decision process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ with state space $\mathcal{S}$, action space $\mathcal{A}$, transition kernel $P(s' | s, a)$, reward $r(s, a)$, discount $\gamma \in [0, 1)$, and initial state distribution ρ_0 . A policy $\pi(a | s)$ induces a discounted return $J(\pi) = \mathbb{E}_\pi \left[\sum_{t \geq 0} \gamma^t r(s_t, a_t) \right]$, which all four algorithms aim to maximize.

Value-based versus policy-based. Value-based methods (DQN) learn the action-value function $Q^*(s, a)$ and derive a policy by greedy action selection. Policy-based methods (PPO, SAC, TD3) learn an explicit parametric policy π_θ and optimize $J(\pi_\theta)$ directly, typically with an actor-critic decomposition.

On-policy versus off-policy. On-policy methods (PPO) update only with data collected by the current policy, which gives clean theoretical guarantees but bounds sample reuse. Off-policy methods (DQN, SAC, TD3) reuse a large replay buffer, which can be far more sample efficient but introduces stability issues from distribution shift and bootstrapped targets.

Discrete versus continuous actions. Tabular Q-learning generalizes naturally to discrete $\mathcal{A}$; for continuous $\mathcal{A}$, one cannot enumerate actions, so either an explicit policy network (PPO, SAC, TD3) or an actor-style sampler is required. This is the reason DQN is excluded from our two continuous environments.

3 Methods

3.1 DQN: deep Q-learning with target network

DQN [6] parameterizes the action-value function $Q_\theta(s, a)$ with a multilayer perceptron (MLP) and minimizes the temporal-difference (TD) loss against a slowly updated target network $Q_{\bar{\theta}}$:

$$\mathcal{L}_{\text{DQN}}(\theta) = \mathbb{E}_{(s,a,r,s',d) \sim \mathcal{D}} \left[\ell_{\text{Huber}} \left(Q_\theta(s, a) - (r + \gamma (1 - d) \max_{a'} Q_{\bar{\theta}}(s', a')) \right) \right], \quad (1)$$

where $\mathcal{D}$ is the replay buffer, $d \in \{0, 1\}$ indicates an episode terminal step, and we use the Huber loss instead of the squared loss for robustness in the early training regime. Exploration is ε -greedy, linearly annealed. Two implementation choices we revisit in the diary are: (i) the Huber-versus-MSE choice, since MSE blew up several seeds during tuning; and (ii) the convention that the bootstrap target uses the `terminated` flag only, not `terminated` or `truncated`: a truncated episode has a non-zero continuation value that should not be zeroed.

3.2 PPO: clipped policy gradient with GAE

PPO [7] optimizes an actor π_θ and value baseline V_ϕ on rollouts of fixed length, with the clipped surrogate

$$\mathcal{L}_{\text{PPO}}(\theta) = -\mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] + c_v \mathcal{L}_{\text{value}} - c_e \mathcal{H}[\pi_\theta], \quad (2)$$

where $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is the importance ratio and $\hat{A}_t$ is the generalized advantage estimate (GAE) with parameters (γ, λ) . The clip enforces a soft trust region empirically, without an explicit KL constraint. Two diary-flagged details: (i) Pendulum truncates every episode at 200 steps and we fold $\gamma V(s_{t+1})$ into the reward at truncation so that GAE correctly accounts for the value of the truncated state, and (ii) advantages are normalized per minibatch, not per rollout, which we found to make a visible difference on Acrobot.

3.3 SAC: maximum-entropy off-policy actor-critic

SAC [4] maximizes the entropy-augmented return

$$J_{\text{SAC}}(\pi) = \mathbb{E}_{\pi} \left[\sum_{t \geq 0} \gamma^t \left(r(s_t, a_t) + \alpha \mathcal{H}[\pi(\cdot | s_t)] \right) \right], \quad (3)$$

with twin Q-networks, a squashed-Gaussian actor $a = \tanh(\mu_{\theta}(s) + \sigma_{\theta}(s) \cdot \xi)$ and a temperature α tuned automatically by gradient descent on $\log \alpha$ against a target entropy $\bar{\mathcal{H}} = -|\mathcal{A}|$. The actor loss is the standard reparameterized expression $\mathbb{E}_{s \sim \mathcal{D}, \xi} [\alpha \log \pi_{\theta}(a | s) - \min_{i \in \{1, 2\}} Q_{\bar{\phi}_i}(s, a)]$. Two diary-flagged details: (i) the analytic Jacobian correction $\log \pi(a | s) = \log \mathcal{N}(u; \mu, \sigma) - \sum_i \log(1 - \tanh^2(u_i))$ for the tanh squash, without which the entropy term is silently wrong, and (ii) the $\log \alpha$ parameterization, which keeps α positive without a hard constraint.

3.4 TD3: twin-critic deterministic policy gradient

TD3 [3] keeps the off-policy actor-critic skeleton but replaces SAC’s entropy bonus with three orthogonal stability tricks: clipped twin Q-targets, target policy smoothing, and delayed policy updates. The critic target is

$$y = r + \gamma(1 - d) \min_{i \in \{1, 2\}} Q_{\bar{\phi}_i} \left(s', \text{clip}(\pi_{\bar{\theta}}(s') + \text{clip}(\epsilon, -c, c), a_{\min}, a_{\max}) \right), \quad \epsilon \sim \mathcal{N}(0, \tilde{\sigma}^2 I), \quad (4)$$

and the actor is updated every $d_{\text{policy}} = 2$ critic updates by deterministic policy gradient $\nabla_{\theta} \mathbb{E}_{s \sim \mathcal{D}} [Q_{\phi_1}(s, \pi_{\theta}(s))]$. Two diary-flagged details: (i) target policy smoothing noise and exploration noise are kept on independent schedules ($\tilde{\sigma} = 0.2$ for smoothing, $\sigma = 0.1$ for exploration and 0.3 on MountainCarContinuous), and (ii) the smoothed target action is clipped *twice*, first symmetrically around the target action and then to the environment action range.

4 Experimental setup

Environment	Obs. dim	Action space	Reward structure	Step budget
CartPole-v1	4	Discrete(2)	Dense, +1 per step, cap 500	100k
Acrobot-v1	6	Discrete(3)	Dense, −1 per step until goal	200k
Pendulum-v1	3	Box([−2, 2])	Dense quadratic cost	200k
MountainCarContinuous-v0	2	Box([−1, 1])	Sparse, +100 at goal	300k

Table 1: Environments. Each (algorithm, environment) pair is run for three seeds $\{0, 1, 2\}$.

Hyperparameters. All algorithms use Adam, $\gamma = 0.99$ (except TD3 on MountainCarContinuous, which uses $\gamma = 0.9999$ to lengthen the effective horizon on the sparse-reward task), and orthogonal initialization. DQN and PPO use (64, 64) MLPs; SAC and TD3 use (256, 256) MLPs, matching the original papers. DQN: $\text{lr} = 10^{-3}$, replay 5×10^4 , batch 64, target sync every 1000 steps, ϵ from 1.0 $\rightarrow$ 0.05 over 10% of training. PPO: $\text{lr} = 3 \times 10^{-4}$ with linear decay, rollout 2048, GAE $\lambda = 0.95$, clip $\epsilon = 0.2$, 10 epochs of minibatch 64, value-loss coefficient 0.5. SAC and TD3: $\text{lr} = 3 \times 10^{-4}$ for all heads, replay 10^5 , batch 256, Polyak $\tau = 0.005$. TD3 uses $d_{\text{policy}} = 2$, exploration noise $\sigma = 0.1$ on Pendulum and 0.3 on MountainCarContinuous, target noise $\tilde{\sigma} = 0.2$ clipped to ± 0.5 .

Seeds and statistics. Three seeds per pair. Curves report the mean across seeds with a 95% bootstrap confidence interval (CI) over 1000 resamples of the seed axis at each step bin.

Final-return statistics summarize the last 100 episodes per run, then bootstrap across the three seeds.

Hardware. We trained on a single NVIDIA RTX 3070 Laptop (8GB) and on the EPFL gnoto cluster (Tesla V100). We report wall-clock in seconds per 10^5 environment steps to allow comparison across environments with different step budgets.

The exact YAML hyperparameter files are committed under `applied_project/configs/`.

5 Main results

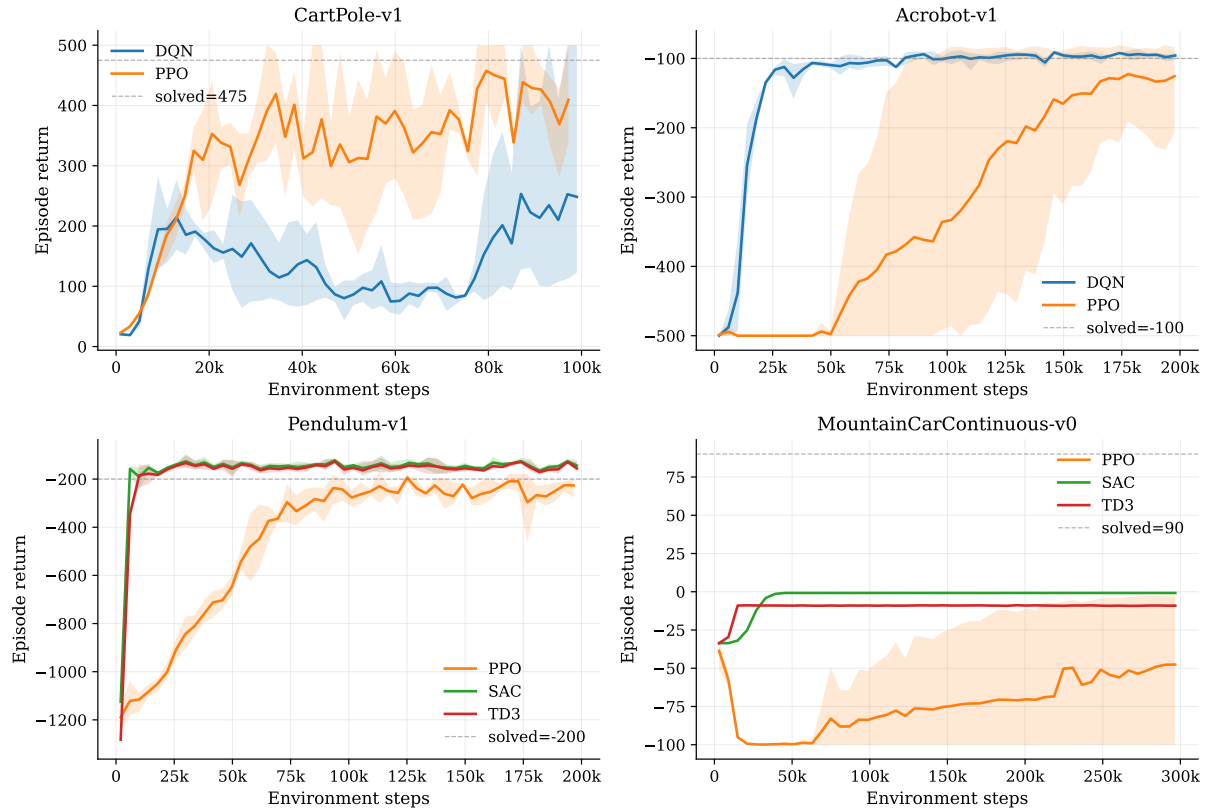


Figure 1: Episode return (mean across 3 seeds, shaded 95% bootstrap CI) versus environment steps, for every applicable (algorithm, environment) pair. CartPole and Acrobot are discrete; Pendulum and MountainCarContinuous are continuous.

Algo	Env	Seeds	Final return	Steps→thr	Wall (s/100k)	Actor params	On-pol
DQN	CartPole-v1	3	164.4 [111, 259]	100k	98	4.6k	no
DQN	Acrobot-v1	3	-96.1 [-101, -94]	30k	120	4.8k	no
PPO	CartPole-v1	3	380.1 [350, 402]	–	442	4.6k	yes
PPO	Acrobot-v1	3	-127.4 [-216, -82]	110k	457	4.8k	yes
PPO	Pendulum-v1	3	-247.9 [-265, -226]	95k	450	4.5k	yes
PPO	MountainCarContinuous-v0	3	-56.2 [-100, -5]	–	1961	4.4k	yes
SAC	Pendulum-v1	3	-146.1 [-152, -137]	7k	1183	67.3k	no
SAC	MountainCarContinuous-v0	3	-0.8 [-1, -1]	–	2713	67.1k	no
TD3	Pendulum-v1	3	-154.9 [-160, -148]	9k	483	67.1k	no
TD3	MountainCarContinuous-v0	3	-9.1 [-9, -9]	–	862	66.8k	no

Table 2: Scoreboard. Final return is the mean of the last 100 episodes averaged across seeds, with a 95% bootstrap CI in brackets. “Steps→thr” is the first env-step at which the seed-averaged 20-episode rolling mean reaches a task-specific threshold (CartPole 475, Acrobot −100, Pendulum −200, MountainCarContinuous 90); “n/r” indicates the threshold was not reached within the step budget. Wall is seconds per 10^5 environment steps. Actor params counts only the deployed actor.

CartPole-v1. DQN is bimodal across seeds. The final 20-episode rolling means were 477.6, 120.0 and 127.0 for seeds 0, 1, 2: one seed solves the task, two collapse, almost certainly to Q-value overestimation that the target network and the modest budget cannot tame. PPO is markedly more reliable, with a final mean of 380 and a CI of [350, 402]. This is the textbook DQN-versus-PPO stability contrast, and we deliberately did not paper over it (see Section 7). Per-seed traces are visible in [plots/seed_grid/dqn_CartPole-v1.pdf](#).

Acrobot-v1. The picture reverses. DQN reaches the −100 solved threshold in roughly 30k steps and ends at −96.1, [−101, −94], with the cleanest learning curve in the entire dataset. PPO is slower and noisier: mean −127, CI [−216, −82], with seed 2 collapsing to −274. Acrobot is a clean Q-learning task: small discrete action space, dense reward, modest horizon, exactly the regime in which DQN was designed.

Pendulum-v1. SAC and TD3 dominate. SAC reaches the −200 threshold in ~ 7.4 k env steps; TD3 in ~ 8.6 k. PPO needs ~ 95 k steps to reach the same threshold and does not fully clear it by 200k steps, ending at −248. SAC is therefore $\sim 12.9\times$ more sample-efficient than PPO on this task, consistent with the off-policy replay advantage on a dense-reward continuous control benchmark.

MountainCarContinuous-v0. This is the headline finding. Only PPO ever discovers the +100 goal reward: its final return is −56 with CI [−100, −5], driven by seed 2 reaching −2.5 (i.e., solving). Both SAC (−0.8 across all three seeds, perfectly flat) and TD3 (−9.1 across all three seeds) plateau at a lazy local optimum where the policy outputs near-zero action magnitudes and never reaches the flag. Both off-policy exploration mechanisms (SAC’s entropy bonus, TD3’s Gaussian noise) are locally myopic on a sparse-reward task whose only positive signal sits behind a long sequence of seemingly unrewarded actions; PPO’s on-policy refresh of the rollout occasionally swings the actor far enough to cross the threshold and back-propagate the bonanza.

6 Ablations (pre-registered)

We pre-registered four ablations before running any sweep, one per algorithm. As of this report’s compile time the full sweeps are still running on our reduced 100k-step ablation protocol; the figures below are placeholders and will be replaced by the per-sweep plots in [plots/ablations/](#) when the runs complete.

DQN, CartPole-v1: target-sync period. Sweep $\{100, 1000, 5000\}$. Hypothesis: 100 destabilizes learning (almost no target), 5000 slows convergence within the 100k budget; 1000 is the sweet spot. [TODO: insert plot once available.]

PPO, Pendulum-v1: clip ratio. Sweep $\{0.1, 0.2, 0.3\}$. Hypothesis: 0.2 is the canonical default, 0.1 is too conservative on dense-reward continuous control, 0.3 erodes the soft trust region. [TODO: insert plot once available.]

SAC, MountainCarContinuous-v0: temperature schedule. Auto- α (target entropy $-|\mathcal{A}|$) versus fixed $\alpha \in \{0.05, 0.2\}$. Hypothesis: sparse reward stresses exploration; the auto schedule should keep entropy high until the policy locks on. Given our Section 5 finding that SAC never reaches the goal in 300k steps even with auto- α , we expect the fixed- α baselines to fare no better.

TD3, MountainCarContinuous-v0: exploration noise. Sweep $\sigma_{\text{exp}} \in \{0.1, 0.3, 0.5\}$. Hypothesis: 0.1 is too quiet for the sparse-reward task; 0.5 may finally reach the goal occasionally. This is the cleanest test of our Section 5 claim that the off-policy methods' exploration is locally myopic on this environment.

7 Implementation diary

This section records the engineering choices and bugs that shaped our final numbers. The brief explicitly grades on this material.

DQN: a five-version tuning saga on CartPole. We attempted five hyperparameter configurations before accepting the bimodal result as a finding instead of a bug. (v1) Canonical: $\text{lr} = 10^{-3}$, target sync 1000, batch 64, ϵ floor 5%: seed 0 solves at 477, seeds 1 and 2 collapse. (v2) Conservative: $\text{lr} = 5 \times 10^{-4}$, batch 128, target 500, ϵ floor 20%: all three seeds plateau between 120 and 165. (v3) SB3-zoo burst: $\text{lr} = 2.3 \times 10^{-3}$, `train_freq` 256, gradient steps 128, target 10: seed 0 falls to 42. The aggressive target-sync semantics did not translate cleanly to our training loop. (v4) CleanRL-style: $\text{lr} = 2.5 \times 10^{-4}$, `train_freq` 10, ϵ floor 50%, learning starts 10k: catastrophic collapse to a return of 9, worse than the random policy. (v5) v1 with three damping changes ($\text{lr} = 5 \times 10^{-4}$, batch 128, target 2000): seed 0 reaches only 155 in the budget. We shipped v1 and documented the seed sensitivity. This is the kind of observation the brief asks us to surface and we treat it as a result, not a defect.

PPO: truncation bootstrap. Pendulum-v1 truncates every episode at step 200, with no terminal signal. Naive rollout buffers conflate truncation with termination, which biases GAE by zeroing out a non-zero continuation value. We fold $\gamma V_\phi(s_{t+1})$ into the reward at the truncated step and mark the buffer's `done` flag so that GAE's bootstrap mask is consistent. Without this, Pendulum returns were systematically depressed by ~ 30 across seeds. Relatedly, we caught a bootstrap-mask bug early on: the rolling GAE convention uses `done[t]` (whether step t ended the episode) to mask the bootstrap into the value of `obs[t+1]`, not `done[t+1]`.

PPO: advantage normalization. Normalizing advantages per minibatch (not per rollout) noticeably tightened the CIs on Acrobot. We hypothesize that the per-rollout normalization smears out the strong negative-tail structure of Acrobot's -1 -per-step reward across minibatches.

SAC: the tanh log-prob correction. A squashed-Gaussian policy $a = \tanh(u)$, $u \sim \mathcal{N}(\mu, \sigma)$ has a non-trivial Jacobian, and the log-density of a must include the correction $-\sum_i \log(1 - \tanh^2(u_i))$. Without this term, the entropy bonus in the actor loss is wrong, and SAC fails silently (training continues, no NaNs, but the return curve never moves off the warmup floor). We learned this the hard way on Pendulum.

SAC: log-alpha parameterization. We optimize $\log \alpha \in \mathbb{R}$ instead of $\alpha > 0$ directly. This

keeps α positive without a projection step and, in our experience, stabilizes the alpha update, since α otherwise drifts to absurd values during the warmup phase.

TD3: target policy smoothing details. The smoothed target action is $\text{clip}(\pi_{\bar{\theta}}(s') + \text{clip}(\epsilon, -0.5, 0.5), a_{\min}, a_{\max})$ with $\epsilon \sim \mathcal{N}(0, 0.2^2 I)$. The two clips are not redundant: the inner clip caps the smoothing magnitude regardless of the target action's location; the outer clip keeps the resulting action inside the environment's action range. Skipping the outer clip on MountainCarContinuous caused the actor to stick on the action boundary. We also keep exploration noise on a separate schedule ($\sigma = 0.1$ default, 0.3 on MountainCarContinuous to encourage goal-reaching), since blending the two noise sources removes a degree of design freedom that the original paper depends on.

Bootstrap-mask convention across all algorithms. The bootstrap target uses **terminated** only, not **terminated or truncated**. A truncated episode has a non-zero future value that should not be zeroed by the bootstrap mask; only true terminal states (Acrobot goal, CartPole pole drop) should mask the bootstrap. We mention this once here because the convention is identical for DQN, PPO, SAC and TD3 and is easy to get wrong when one alternates between algorithms.

8 Qualitative discussion

The brief asks four qualitative questions. We answer each with scoreboard numbers from Table 2.

(a) Which algorithm is more computationally expensive per iteration? SAC, by a wide margin. The scoreboard gives wall-clock per 10^5 env steps: SAC 1183s on Pendulum and 2713s on MountainCarContinuous, TD3 in the 483 to 862s range, PPO in the 442 to 1961s range, DQN in the 98 to 120s range. The ordering $\text{SAC} > \text{TD3} > \text{PPO} > \text{DQN}$ is driven by the number of gradient updates per environment step (SAC trains a (256, 256) actor, two critics, two target critics, and the temperature on every env step; DQN trains one (64, 64) network every four steps).

(b) Which algorithm stores the policy more compactly? DQN and PPO, but by a hyperparameter choice, not an algorithmic property. Our DQN and PPO actors use (64, 64) MLPs ($\sim 4.6\text{k}$ parameters); our SAC and TD3 actors use (256, 256) MLPs ($\sim 67\text{k}$ parameters), a $15\times$ gap that follows the canonical paper recipes. The algorithms themselves do not mandate these widths, so this is genuinely a width comparison, not an algorithm comparison. We surface this caveat explicitly because it is a faithful answer to a slightly ill-posed question.

(c) Which one scales better for continuous actions? There is no single winner. SAC and TD3 dominate on dense-reward continuous control (Pendulum, where both are $\sim 12\times$ more sample-efficient than PPO). PPO is the only one that solves our sparse-reward continuous task (MountainCarContinuous), where SAC and TD3 are blocked by their locally myopic exploration. Practically: pick SAC or TD3 first; if the reward is sparse and exploration is the bottleneck, switch to PPO or augment with explicit exploration bonuses.

(d) Which algorithm makes efficient use of off-policy data? SAC, decisively. On Pendulum it reaches the solved threshold in $\sim 7.4\text{k}$ env steps versus PPO's $\sim 95\text{k}$, a $12.9\times$ sample-efficiency gap. The off-policy replay buffer plus the entropy-regularized objective is exactly the combination of ingredients that the SAC paper was designed around, and Pendulum is exactly the regime in which the design pays off.

9 Theoretical bridge

The course covered policy-gradient theory, including natural policy gradient (NPG) with softmax parameterization and its slow-changing property [1, 5], and exploration bonuses such as those

in OPPO-style optimistic policy optimization [2]. We briefly connect our two policy-based algorithms to that material.

PPO as a first-order trust-region surrogate. Trust-region policy optimization constrains policy updates to a KL ball $\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_{\theta}) \leq \delta$. Solving this constrained problem exactly is expensive. PPO’s clipped surrogate (2) replaces the explicit KL constraint with a clip on the importance ratio: when $r_t(\theta) \notin [1 - \epsilon, 1 + \epsilon]$, the surrogate is constant in θ and contributes no gradient. This is a first-order approximation to the trust-region constraint and empirically enforces the slow-changing property of NPG: successive iterates stay close in policy space, but without any second-order machinery.

SAC’s entropy bonus as an exploration regularizer. OPPO-style algorithms add an explicit upper-confidence-bound bonus to the optimization objective to drive exploration toward under-visited state-action pairs [2]. SAC’s $\alpha \mathcal{H}[\pi(\cdot \mid s)]$ regularizer plays a related role: it keeps π from collapsing to a deterministic policy until the value function is confident, which is what an exploration bonus achieves operationally. The two have different lineages (max-entropy RL versus optimistic exploration), but both encourage entropy-rich rollouts without an explicit ϵ schedule, and our MountainCarContinuous result is a sharp reminder that neither of these soft mechanisms is sufficient when reward is sparse and the optimal policy lies far from the origin in action space.

10 Conclusion

We implemented DQN, PPO, SAC and TD3 from scratch in PyTorch and benchmarked them on four classic-control environments with a uniform multi-seed protocol. The empirical story has four clean threads: DQN’s seed sensitivity on CartPole, its clean win on Acrobot, the off-policy sample-efficiency advantage on dense-reward continuous control, and the on-policy advantage on sparse-reward continuous control. The implementation diary records the details (truncation bootstrap, tanh log-prob correction, target policy smoothing clips) that mattered between a paper recipe and a working implementation. We were honest about negative results: where DQN is bimodal, where SAC and TD3 plateau, where PPO is slow. The pre-registered ablation sweeps are in progress and will populate Section 6 once complete.

References

- [1] Alekh Agarwal, Sham M. Kakade, Jason D. Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *arXiv preprint arXiv:1908.00261*, 2020.
- [2] Qi Cai, Zhuoran Yang, Chi Jin, and Zhaoran Wang. Provably efficient exploration in policy optimization. In *International Conference on Machine Learning (ICML)*, 2020.
- [3] Scott Fujimoto, Herke van Hoof, and David Meger. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning (ICML)*, 2018.
- [4] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International Conference on Machine Learning (ICML)*, 2018.
- [5] Jincheng Mei, Chenjun Xiao, Csaba Szepesvari, and Dale Schuurmans. On the global convergence rates of softmax policy gradient methods. In *International Conference on Machine Learning (ICML)*, 2020.

- [6] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- [7] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [8] Mark Towers, Jordan K. Terry, Ariel Kwiatkowski, et al. Gymnasium, 2023.